🧠 LeetCode: Construct Product Matrix (Deep Notes)

📌 Problem Summary

Given a 2D grid, for each cell, compute the product of all other cells modulo **12345**.

🌱 First Intuition (Natural but Wrong)

> Compute total product → divide by current element

```
result[i][j] = total_product / grid[i][j]
```

Why this feels correct

- Works perfectly in normal arithmetic
- Easy to implement
- Handles most cases except edge cases like zero

❌ Critical Mistake

🚨 Division under modulo

You used:

```
mul = (mul * grid[i][j]) % 12345
res[i][j] = mul / grid[i][j]
```

This is **incorrect**.

❗ Key Rule

> Division does NOT work under modulo unless you use modular inverse

And modular inverse: - Only exists when `gcd(x, MOD) == 1` - MOD = 12345 (not prime) → many numbers don't have inverse

So this approach breaks mathematically.

🧠 Correct Insight

> Avoid division entirely

This problem is actually:

👉 **"Product of Array Except Self"** (classic pattern)

🔑 Core Idea

For each index:

```
result[i] = product(left of i) * product(right of i)
```

✨ Approach

Step 1: Flatten matrix

Convert 2D → 1D array

Step 2: Prefix product

```
prefix[i] = product of elements before i
```

Step 3: Suffix product

```
suffix[i] = product of elements after i
```

Step 4: Combine

```
result[i] = (prefix[i] * suffix[i]) % MOD
```

🧩 Why This Works

- Uses only multiplication → safe under modulo
- Avoids division completely

- Works for all edge cases (including zeros)

⚠️ Your Mistakes (Important for Revision)

1. Ignored modulo implications

- Saw `% 12345` but treated it like normal arithmetic
- Didn't question whether division is valid

---

2. Jumped to coding too fast

- Idea → immediately implemented
- No validation of math correctness

---

3. Didn't recognize pattern

- This is a direct variant of:
- "Product of Array Except Self"

---

4. Over-focused on edge cases (zeros)

- Tried to fix symptoms instead of fixing core approach

🧠 Pattern to Remember

Whenever you see:

- "Product of all elements except current"
- "No division allowed" OR "modulo present"

👉 Think immediately:

> Prefix + Suffix product

🪶 Mental Checklist for Future Problems

Before coding, pause and ask:

1. Can I divide safely?
2. Is modulo involved?
3. Does this resemble a known pattern?

🧭 Key Takeaway

> The mistake was not logic—it was applying the right idea in the wrong mathematical context.

Once you learn to: - respect constraints - recognize patterns

These problems become straightforward.

🚀 Next Step (Optional Improvement)

You can optimize space: - Use output array instead of prefix/suffix arrays - Do two passes (left → right, right → left)

End of Notes